Name: Abdelrahman Hisham Ismail
Member ID: AI2617

IEEE SSCS AUSC

Task: Matrix Normalization in AI

Date: February 2026

1. Introduction: Why Normalize? 3
2. Core Mathematical Foundations 3
3. Taxonomy in Deep Neural Networks 3
4. Advanced Applications: GNNs and Vector DBs 4
5. Real-World Case Study: Pinterest 4

1. Question 1: Why is NumPy significantly faster than pure Python loops?

NumPy's massive speed advantage over pure Python loops boils down to three main factors:

- Vectorization and C-Backend: When you use a Python for loop, Python's interpreter processes the code line-by-line, which is slow. NumPy delegates operations to highly optimized, pre-compiled C code behind the scenes. It performs operations on entire arrays at once (vectorization) rather than stepping through one element at a time.
- Memory Contiguity: Python lists are essentially arrays of pointers pointing to scattered locations in your computer's memory. NumPy arrays store elements of the same type in one continuous, unbroken block of memory. This makes fetching and processing the data exponentially faster for the CPU.
- No Type Checking Overhead: Python is dynamically typed, meaning in a standard loop, it checks the data type (integer, float, string) of *every single item* before operating on it. NumPy arrays are statically typed (all elements must be the same type), so it completely skips this time-consuming step.

2. Question 2: Multicollinearity in the Normal Equation

2.1. What mathematical problem can occur?

The matrix $(X^T X)$ becomes singular (non-invertible) or very close to it (ill-conditioned). Because the Normal Equation requires calculating the inverse, $(X^T X)^{-1}$, the calculation will either throw an error or result in wildly unstable, inaccurate θ values.

2.2. Why does this happen?

In linear algebra, if features are highly correlated or perfectly duplicated, it means the columns of your matrix X are linearly dependent (one column can be perfectly predicted by another). When a matrix has linearly dependent columns, its determinant drops to zero. Mathematically, you cannot divide by zero, meaning the inverse of $(X^T X)$ simply does not exist.

2.3. How can it be solved without removing features?

We have two great options to solve this mathematically:

1. Regularization (Ridge Regression): You add a tiny "penalty" matrix (the identity matrix I multiplied by a small number λ) to the diagonal of $(X^T X)$. This forces the matrix to become mathematically invertible again. The modified equation looks like this:
$$\theta = (X^T X + \lambda I)^{-1} X^T y$$
2. Use the Pseudoinverse: Instead of calculating the strict mathematical inverse (using np.linalg.inv), you can calculate the Moore-Penrose Pseudoinverse (using np.linalg.pinv in NumPy). The pseudoinverse uses Singular Value Decomposition (SVD) to safely bypass the singularity issue and find the best approximate solution.